

Fundamental Event Scanner Pro

Manual de Despliegue a GitHub Pages

Con énfasis en seguridad de API keys

Lo que es DIFERENTE en este manual vs Stochastic Scanner Pro:

FESP usa **API keys de Alpha Vantage y Marketstack**. Esto requiere cuidado especial al deploy a GitHub. Si haces lo incorrecto, expones tus keys públicamente y alguien puede:

- Consumir tu cuota diaria/mensual de Alpha Vantage
- **Generar cobros en tu tarjeta** (overage de Marketstack)
- Acceder a tu cuenta y modificar configuración

Este manual te guía paso a paso para hacerlo bien.

RESULTADO FINAL: Dashboard FESP publicado en una URL pública, actualizándose diariamente con datos frescos via GitHub Actions, con tus API keys completamente seguras.

Tiempo: 30-50 min primera vez · **Costo:** \$0 · **Pre-req:** ZIP de FESP listo, cuenta GitHub

Contenido

1. Reglas críticas de seguridad antes de empezar
2. Pre-requisitos (cuenta GitHub, API keys regeneradas)
3. Verificar que .env y .gitignore están correctos
4. Crear el repositorio en GitHub
5. Subir el proyecto (3 caminos: Web / CLI / Desktop)
6. Configurar GitHub Secrets para las API keys
7. Activar GitHub Pages
8. Activar y configurar GitHub Actions
9. Primer build manual con datos reales
10. Editar tu watchlist desde GitHub Web
11. Monitorear el uso de cuota AV/Marketstack
12. Qué hacer si expones una API key por error
13. Mantenimiento periódico
14. Troubleshooting de errores comunes
15. Checklist final

1. Reglas críticas de seguridad

Lee esta sección dos veces antes de continuar. Las API keys son contraseñas. Trátalas con el mismo cuidado.

Las 5 reglas absolutas:

Regla 1 — NUNCA pongas API keys en código fuente

■ NO hagas esto:

```
$ AV_KEY = "KNHVTBYX79H4ECIG" # MAL — esto se subiría a GitHub
```

✓ Sí haz esto:

```
$ AV_KEY = os.environ.get("ALPHA_VANTAGE_API_KEY") # BIEN
```

Regla 2 — NUNCA commits del archivo .env

El .env contiene tus keys reales. Solo .env.example (template sin keys reales) debe subirse a GitHub. El .gitignore del proyecto ya bloquea .env.

Regla 3 — NUNCA pegues keys en chat, ticket, screenshot

Si alguna vez expones una key (incluso en chat privado, email, captura de pantalla, log compartido), **regénérala inmediatamente**. Ver sección 12.

Regla 4 — Usa GitHub Secrets para Actions

Las GitHub Actions necesitan las keys para correr el build diario. Pero NO se ponen en el código del workflow. Se configuran como 'Secrets' en Settings → Secrets and variables → Actions. GitHub las inyecta como variables de entorno solo durante la ejecución.

Regla 5 — Repos públicos no son problema SI haces lo anterior

GitHub Pages gratis requiere repo público. Eso significa que cualquiera puede ver tu código. ESTÁ BIEN — porque tu código no contiene keys (siguiendo reglas 1-4). Lo que queda público es lógica y estructura, no credenciales.

2. Pre-requisitos

2.1 Cuenta de GitHub

Si no tienes, crea una en github.com/signup. Es gratis. Elige un username profesional — será parte de tu URL pública (ej: `otarazona88.github.io`).

2.2 API keys regeneradas (RECOMENDADO)

Si en algún momento pegaste tus API keys en chat (este chat o cualquier otro), o las compartiste en captura de pantalla, **regénalas ANTES de subir el proyecto**. Es más seguro asumir que están comprometidas.

Cómo regenerar:

- **Alpha Vantage:** <https://www.alphavantage.co/support/#api-key> — el botón 'GET FREE API KEY' regenera (tiraán la anterior). Toma 30 segundos.
- **Marketstack:** <https://marketstack.com/dashboard> → 'My Account' → 'Reset API Key'. Toma 30 segundos.

2.3 ZIP del proyecto descomprimido

Descomprime `fundamental-event-scanner-pro-v1.0.zip` en una carpeta local (ej: tu Escritorio). Verás:

Archivo / Carpeta	Va a GitHub?
<code>index.html</code>	✓ Sí
<code>README.md</code>	✓ Sí
<code>requirements.txt</code>	✓ Sí
<code>watchlist.txt</code>	✓ Sí (sin tus keys, solo tickers)
<code>.env.example</code>	✓ Sí (template, no contiene keys reales)
<code>.gitignore</code>	✓ Sí (CRÍTICO — protege <code>.env</code>)
<code>scripts/</code> (todos los <code>.py</code>)	✓ Sí
<code>.github/workflows/refresh.yml</code>	✓ Sí
<code>data/snapshot.json</code>	✓ Sí (demo data)
<code>.env</code>	✗ NO — contiene tus keys reales
<code>.cache/</code>	✗ NO — caché local
<code>__pycache__/</code>	✗ NO — bytecode Python

El `.gitignore` incluido en el proyecto ya está configurado para proteger automáticamente `.env`, `.cache`, `__pycache__`, y otros archivos sensibles. Verifica en sección 3.

3. Verificar .env y .gitignore

Antes de subir cualquier cosa a GitHub, dos verificaciones críticas:

3.1 Verificar que .env NO está en la carpeta

Tu archivo .env (con keys reales) **debe vivir solo en tu PC**, nunca en el ZIP que subes a GitHub. Pero si accidentalmente lo pusiste ahí, hay que sacarlo:

En Linux/macOS:

```
$ cd ~/Desktop/fundamental-event-scanner-pro
$ ls -la | grep -E '^\.env'
```

Si ves .env en la lista, BORRALO antes de subir:

```
$ rm .env # solo si es una copia que no necesitas
```

En Windows (PowerShell):

```
$ cd C:\Users\TuUsuario\Desktop\fundamental-event-scanner-pro
$ Get-ChildItem -Force | Where-Object Name -EQ '.env'
```

3.2 Verificar contenido de .gitignore

Abre .gitignore en un editor. Debe incluir al menos estas líneas:

```
$.env
$.env.local
$.env.*.local
$.cache/
$__pycache__/*
$*.pyc
```

Si abres .gitignore y NO ves estas líneas, agrégalas manualmente. Es la barrera principal contra exposición de keys.

3.3 Verificar que .env.example NO contiene keys reales

Abre .env.example. Debe tener placeholders, NO tus keys reales:

```
$ # CORRECTO:
$ ALPHA_VANTAGE_API_KEY=your_alpha_vantage_key_here
$ MARKETSTACK_API_KEY=your_marketstack_key_here
```

Si por algún motivo .env.example tiene tus keys reales, REEMPLÁZALAS por placeholders antes de subir.

4. Crear el repositorio

Paso 1 *Login en github.com*

Ve a `github.com` y haz login.

Paso 2 *Click en '+' arriba a la derecha → 'New repository'*

Aparece el formulario de creación.

Paso 3 *Llenar el formulario*

Repository name: `fundamental-event-scanner-pro`

Description: Multi-source fundamental + event-study + MC system

Public/Private: seleccionar **Public**

Initialize with: NO marques ninguna casilla

Paso 4 *Click 'Create repository' (botón verde)*

5. Subir el proyecto

Tres opciones — elige según tu comodidad técnica:

Opción A — Subida manual desde GitHub Web (más fácil)

Paso A1 *En la página del repo recién creado, click en 'uploading an existing file'*

Aparece como link en blanco arriba.

Paso A2 *Arrastra TODOS los archivos del proyecto descomprimido*

Selecciona todo dentro de la carpeta fundamental-event-scanner-pro/ (no la carpeta misma, sino su contenido). **Atención: incluye la carpeta oculta .github** — en Windows activa 'Mostrar archivos ocultos' en el explorador.

NO arrastres .env — solo .env.example.

Paso A3 *Esperar la subida (1-2 minutos)*

Paso A4 *En el campo 'Commit changes', escribir*

Initial commit – FESP v1.0

Paso A5 *Click 'Commit changes' (botón verde)*

Verificación crítica después de subir:

Recorre la lista de archivos del repo. Asegúrate que **.env NO está en la lista**. Si está, contáctame para recuperar — hay que regenerar las keys y limpiar el historial git.

Opción B — Desde línea de comandos (más rápido)

```
$ cd ~/Desktop/fundamental-event-scanner-pro

$ # Verificar que .env NO está aquí (debería NO listarlo):

$ ls -la | grep .env$

$ # Si ves .env, primero borra o renombra antes de continuar

$ git init

$ git add .

$ # Verificar QUÉ se va a subir (NO debe incluir .env):

$ git status

$ # Si .env aparece, ABORTA – revisa .gitignore primero

$ git commit -m "Initial commit – FESP v1.0"

$ git branch -M main

$ git remote add origin
https://github.com/TU-USUARIO/fundamental-event-scanner-pro.git

$ git push -u origin main
```

Opción C — GitHub Desktop (interfaz gráfica)

Sigue las secciones 5.1-5.6 del manual de Stochastic Scanner Pro (los pasos son idénticos). La única diferencia es: asegúrate que `.env` NO esté en la carpeta antes de hacer commit.

6. Configurar GitHub Secrets

Esta sección es la más importante del manual. Aquí es donde le das a GitHub Actions tus API keys de manera SEGURA, sin exponerlas en el código.

Cómo funcionan los Secrets:

Los GitHub Secrets son variables de entorno encriptadas almacenadas por GitHub. Cuando un workflow corre, GitHub las inyecta como variables de entorno solo durante la ejecución. Después se borran de la memoria. **Nunca aparecen en el código, en logs, ni en commits.**

Pasos para configurar:

Paso 1 En tu repo, click 'Settings' (pestaña arriba a la derecha)

Paso 2 En la barra lateral izquierda: 'Secrets and variables' → 'Actions'

Paso 3 Click en el botón verde 'New repository secret'

Aparece un formulario.

Paso 4 Configurar el primer secret

Name: ALPHA_VANTAGE_API_KEY (exacto, sensible a mayúsculas)

Secret: pega tu key real de Alpha Vantage

Click 'Add secret'.

Paso 5 Repetir para Marketstack

Click otra vez 'New repository secret'.

Name: MARKETSTACK_API_KEY

Secret: pega tu key real de Marketstack

Click 'Add secret'.

Paso 6 Verificar

Después de agregar ambos, deberías ver en la lista:

- ALPHA_VANTAGE_API_KEY · Updated [now]
- MARKETSTACK_API_KEY · Updated [now]

GitHub NO te muestra los valores (es por diseño — solo se pueden agregar o sobrescribir, nunca leer).

Cómo se usan en el workflow:

El archivo `.github/workflows/refresh.yml` ya está configurado para usar los secrets así:

```
$ env:

$ ALPHA_VANTAGE_API_KEY: ${ secrets.ALPHA_VANTAGE_API_KEY }

$ MARKETSTACK_API_KEY: ${ secrets.MARKETSTACK_API_KEY }
```

La sintaxis `${ secrets.NAME }` es como GitHub inyecta los secrets. Si no los configuraste en Settings, vendrán como string vacío y el script funcionará solo con Yahoo (graceful fallback).

7. Activar GitHub Pages

Paso 1 *Settings → Pages (en barra lateral)*

Paso 2 *En 'Source', seleccionar 'Deploy from a branch'*

Paso 3 *Configurar branch y folder*

Branch: main

Folder: / (root)

Paso 4 *Click 'Save'*

GitHub empieza a construir tu sitio. Tarda 1-3 minutos.

Verificar:

Después de 2-3 minutos, recarga la página de Pages. Verás:

```
$ Your site is live at https://TU-USUARIO.github.io/fundamental-event-scanner-pro/
```

Click en esa URL. Deberías ver el dashboard FESP funcionando con el demo data (20 tickers sintéticos). El SEMÁFORO SUMMARY arriba muestra: TICKERS=20, GRADE A=0, GRADE B+=4, etc.

8. Activar GitHub Actions

8.1 Habilitar Actions

Paso 1 *Pestaña 'Actions' del repo*

Si está deshabilitado, click 'I understand my workflows, go ahead and enable them'.

Paso 2 *Verificar que aparece el workflow*

Deberías ver 'Refresh FESP Snapshot' en la lista.

8.2 Permisos de escritura (CRÍTICO)

Paso 1 *Settings → Actions → General*

Paso 2 *Sección 'Workflow permissions'*

Casi al final de la página.

Paso 3 *Seleccionar 'Read and write permissions'*

Marcar también 'Allow GitHub Actions to create and approve pull requests'.

Paso 4 *Click 'Save'*

Sin este paso, el workflow NO podrá hacer commit del nuevo snapshot.json y fallará con 'Permission denied'.

8.3 Configurar el modo de enriquecimiento

El workflow YA viene configurado para tu plan free:

```
$ python scripts/build_data.py --deep-mode lite --deep-top-n 12
```

Esto procesa todo el universo con Yahoo + enriquece los 12 tickers de tu watchlist con AV en modo lite (24/25 reqs AV consumidos). Es óptimo para plan free.

Si quieres cambiarlo:

Paso 1 *En el repo: .github/workflows/refresh.yml*

Click en el archivo, luego ícono lápiz (Edit).

Paso 2 *Cambiar la línea 'run:'*

Modifica los argumentos según necesites:

- `--deep-mode full --deep-top-n 5` (5 tickers, datos completos)
- `--deep-mode fundamentals_only --deep-top-n 25` (25 tickers, solo OVERVIEW)
- `--deep-top-n 0` (sin AV, solo Yahoo)

Paso 3 *Commit los cambios*

El próximo run usará la nueva configuración.

9. Primer build manual

Para no esperar al schedule diario, dispara el workflow ahora:

Paso 1 *Pestaña 'Actions'*

Paso 2 *Click en 'Refresh FESP Snapshot' (en la lista de workflows)*

Paso 3 *Click 'Run workflow' (botón a la derecha)*

Aparece un dropdown. Click 'Run workflow' verde para confirmar.

Paso 4 *Esperar 5-10 minutos*

El workflow descarga ~80 tickers de Yahoo + 12 tickers AV. Verás logs en tiempo real si haces click en el run.

Lo que esperar en los logs:

```
$ Building FESP snapshot
$ Tickers : 84 across 7 groups
$ Deep mode : lite
$ Deep requested : 12
$ Deep actual : 12 (after AV quota check)
$ Watchlist tickers : 12 (AAPL, AMZN, GOOGL, ...)
$ ...
$ [DEEP] [1/84] AAPL... FESP=72 (B+) QS=75 src=yah,yah,alp 2300ms
$ [2/84] BRK-B... FESP=68 (B+) QS=64 src=yah,yah 850ms
$ ...
```

Verificar el resultado:

1. Ve a tu URL pública FESP
2. La fecha 'BUILT' arriba a la derecha debe ser hoy
3. La columna 'TICKERS' debe ser cercana a 84
4. Los tickers de tu watchlist deben tener columnas SENT/INSIDER/NEXT ER pobladas

10. Editar tu watchlist desde GitHub Web

Una de las ventajas de tener el proyecto en GitHub: puedes editar tu watchlist desde cualquier dispositivo (laptop, tablet, móvil) sin clonar el repo.

Paso 1 *En tu repo, click en watchlist.txt*

Paso 2 *Click en el ícono de lápiz (=■) arriba a la derecha del archivo*

Entras en modo edición.

Paso 3 *Editar el archivo*

Comenta/descomenta tickers según tu prioridad. Recuerda los límites de tu plan free:

- Modo lite: máximo 12 tickers descomentados
- Modo full: máximo 5 tickers descomentados

Paso 4 *Scroll abajo, escribir commit message*

Ej: Add CIB and EC to watchlist

Paso 5 *Click 'Commit changes'*

El próximo run del workflow usará tu nueva watchlist.

Tip: si quieres aplicar cambios **inmediatamente** sin esperar al schedule, ve a Actions → Run workflow después de hacer el commit.

11. Monitorear el uso de cuota

Tu plan free tiene límites estrictos. Aprende a monitorearlos:

11.1 Alpha Vantage (25 req/día)

Donde ver:

- Dashboard: <https://www.alphavantage.co/>
- En tus logs locales: `.cache/_api_usage.json`

Si excedes 25/día, las requests siguientes fallan con error tipo 'rate-limit' hasta el reset (medianoche UTC).

11.2 Marketstack (100 req/mes)

CRÍTICO: Marketstack tiene **overage fees automáticos**. Si excedes 100 req/mes, te cobran por cada request adicional.

Donde ver:

- Dashboard: <https://marketstack.com/dashboard> → 'Usage Stats'
- En tus logs locales: `.cache/_api_usage.json`

FESP **YA está diseñado** para no exceder Marketstack — solo lo usa como fallback cuando Yahoo falla. En uso normal consumirás ~5-10 reqs/mes. Pero monitórealo de todos modos.

11.3 Detectar uso anómalo

Si ves consumo MUCHO mayor al esperado:

- ¿Tu key fue expuesta y alguien la usa?
- ¿El workflow está corriendo más veces de lo esperado?
- ¿Algún script local con loop infinito?

Acción si detectas anomalía:

1. **Regenera la API key inmediatamente**
2. Actualiza el GitHub Secret con la nueva key
3. Actualiza tu `.env` local con la nueva key
4. La key vieja queda inutilizable en cuestión de minutos

12. Qué hacer si expones una API key

Si en algún momento expones una key (la pegas en un chat, la subes accidentalmente a GitHub, sale en una captura, está en un log que compartiste), **actúa en los próximos 5 minutos**.

Procedimiento de emergencia:

Caso A — Key de Alpha Vantage expuesta

Riesgo: bajo (solo consumirán tu cuota diaria de 25 reqs).

Acción:

1. Ve a <https://www.alphavantage.co/support/#api-key>
2. Click 'Get free API key' — se genera una nueva (la vieja queda invalidada)
3. Actualiza tu GitHub Secret con la nueva key
4. Actualiza tu .env local

Tiempo total: 1 minuto.

Caso B — Key de Marketstack expuesta

Riesgo: medio-alto (overage fees pueden generar cargos en tu tarjeta).

Acción urgente:

1. Ve a <https://marketstack.com/dashboard>
2. Click 'My Account' → 'Reset API Key'
3. Confirma. Se genera una nueva (la vieja queda invalidada)
4. Revisa 'Usage Stats' por si ya hubo abuso
5. Actualiza GitHub Secret y .env local
6. Si ves cargos pendientes, contacta a Marketstack support

Tiempo total: 2 minutos.

Caso C — Subiste .env a GitHub por error

Riesgo: alto si el repo es público.

Acción:

1. **Asume que las keys ya están comprometidas** (scrapers escanean GitHub público en segundos)
2. Regenera **AMBAS** keys inmediatamente
3. Borra el archivo .env del repo:

```
$ git rm --cached .env
```

```
$ git commit -m "Remove .env (was committed by error)"
```

```
$ git push
```

4. **IMPORTANTE:** el archivo sigue en el historial git. Para limpiarlo completamente:

```
$ git filter-repo --path .env --invert-paths
```

```
$ # o usar BFG Repo-Cleaner — ver docs de GitHub
```

5. Si esto te abruma, lo más simple es: borrar el repo de GitHub completo, regenerar keys, crear repo nuevo.

13. Mantenimiento periódico

13.1 Diariamente (automático)

- La GitHub Action corre cada día a las 21:30 UTC
- Tu dashboard se actualiza solo
- No requiere intervención

13.2 Semanalmente (5 min)

- Revisar logs de Actions de la semana — ¿algún fallo?
- Verificar que la fecha 'BUILT' del dashboard sea reciente
- Editar watchlist.txt según prioridades de la semana

13.3 Mensualmente (10 min)

- Revisar Marketstack 'Usage Stats' — ¿bajo del límite?
- Revisar Alpha Vantage usage promedio diario
- Si necesitas más cobertura, considerar plan pago

13.4 Trimestralmente (30 min)

- Rotar API keys preventivamente (mejor práctica de seguridad)
- Revisar y actualizar tickers en TICKER_GROUPS de build_data.py
- Eliminar tickers que ya no cotizan o cambiaron de símbolo
- Validar Quality Score de tu portafolio actual

14. Troubleshooting

14.1 Workflow falla con 'Permission denied'

Causa: falta el paso 8.2 (Read and write permissions).

Solución: Settings → Actions → General → Workflow permissions → Read and write → Save.

14.2 Workflow corre pero no usa AV (todos los tickers sin SENT/INSIDER)

Causa: los Secrets no están configurados o tienen los nombres incorrectos.

Solución:

1. Settings → Secrets and variables → Actions
2. Verifica que existen **exactamente**:
 - ALPHA_VANTAGE_API_KEY
 - MARKETSTACK_API_KEY
3. Si tienen otros nombres (typos), borrar y recrear

14.3 'Daily quota exhausted' temprano en el día

Causa A: ya consumiste tu cuota desde tu PC local antes del run del workflow. La cuota es por API key, no por máquina.

Solución A: reduce uso local, o cambia a modo lite/fundamentals_only en el workflow.

Causa B: tu key está expuesta y alguien la usa.

Solución B: regenera la key (sección 12).

14.4 Dashboard se ve vacío después del primer build

Causa: el snapshot.json no se commiteó correctamente.

Solución:

1. Ve al repo y verifica que data/snapshot.json existe
2. Si NO existe: revisa los logs del workflow — probablemente falló el step 'Commit and push' (ver 14.1)
3. Si SÍ existe pero el dashboard no carga: hard-refresh (Ctrl+Shift+R o Cmd+Shift+R)

14.5 GitHub Actions se desactivan después de 60 días

Causa: GitHub auto-desactiva workflows en repos sin actividad por 60 días.

Solución: ve a Actions → re-habilita el workflow. También puedes hacer un commit pequeño (ej: editar README) para mantenerlo activo.

15. Checklist final

Antes de dar por terminado el deploy, marca cada item:

- Repo 'fundamental-event-scanner-pro' creado como PUBLIC
- Verifiqué que .env NO está en el repo (búsqueda en archivos)
- .gitignore incluye .env, .cache, __pycache__
- .env.example NO contiene keys reales (solo placeholders)
- Todos los archivos del proyecto subidos (incluyendo carpeta .github)
- GitHub Secret ALPHA_VANTAGE_API_KEY configurado
- GitHub Secret MARKETSTACK_API_KEY configurado
- Settings → Pages activado, branch=main, folder=/ root
- URL pública responde (<https://USER.github.io/fundamental-event-scanner-pro/>)
- Dashboard muestra datos del demo correctamente
- Settings → Actions → workflow permissions = Read and write
- Workflow 'Refresh FESP Snapshot' corrido manualmente con éxito
- Snapshot real generado (BUILT date = hoy, no la del demo)
- Mis tickers de watchlist tienen columnas SENT/INSIDER/NEXT ER pobladas
- Verifiqué uso AV en alphavantage.co (25 reqs consumidos en lite mode)
- Verifiqué uso Marketstack (debe ser bajo, ~5-10 reqs)
- Mi watchlist.txt tiene los tickers que QUIERO seguir
- Sé cómo editar watchlist desde GitHub web
- Sé qué hacer si una API key se expone (sección 12)

Cuando todos estén marcados: tu Fundamental Event Scanner Pro está completamente operativo y seguro. Cada día al cierre del mercado, la GitHub Action actualiza los datos automáticamente.

Fundamental Event Scanner Pro v1.0

Manual de Despliegue Seguro a GitHub Pages

API keys protegidas · Daily auto-refresh · \$0 costo